

Engineering Department Doc, Current General Developer Final Handoff Notes

Engineering Department Handoff Notes | Correctr

1. Document Header

Project name: Correctr

Document title: Engineering Department Doc, Current General Developer Final Handoff Notes

Document type: Engineering Department Handoff Notes

Prepared by role: General Developer

Intended readers: Lead Developer, Project Manager, QA Assistant, future developer chats, future AI/context developer, future data-quality/export developer, future personal-memory developer, future neural-ranker/data-science specialist

Prepared date: 2026-05-12

Recommended file placement:

Correctr/1.Docs/Engineering_Department_Doc_Current_General_Developer_Final_Handoff_Notes.md

Primary source of truth: Correctr_Project_Architecture_and_Workflow_Document.docx

Secondary sources: current codebase state, previous early foundation handoff, user test results, latest pytest results, recent correction-event review output, Lead Developer workstream assignments.

2. Purpose and Handoff Statement

This handoff exists because the current General Developer phase has reached a meaningful transition point. Correctr is no longer only an architecture concept or starter repo. It now has a working hotkey/clipboard proof-of-concept, local dictionary correction, structured correction results, SQLite event storage, manual correction capture, AI/context provider interfaces, local Ollama model support, controlled AI routing, local model evaluation tooling, and a review/teach-back workflow for bad corrections.

This document covers the work completed by the current General Developer across the early implementation and local-AI foundation phase. It is intended to help the Lead Developer, Project Manager, QA Assistant, and future developer chats continue cleanly without rereading the full chat history.

This document does **not** replace the architecture document. The architecture document remains the source of truth for project purpose, long-term component design, risk boundaries, and governance. This handoff records the current engineering state and the practical implementation details needed for continuation.

This is not a user-facing README, not a portfolio writeup, and not a final product report. It is an internal engineering continuity document.

3. Executive Technical Status

What works now

- python app.py runs the Correctr hotkey app.

- The user can highlight text, press the configured hotkey, copy selected text into Python, correct it, paste the corrected text back, and save correction events.
- The hotkey and clipboard proof-of-concept has been confirmed in real app testing.
- Local dictionary correction works for a controlled known-typo set.
- `CorrectionResult` provides structured correction output.
- SQLite correction event storage works.
- Manual correction capture works through a terminal script.
- Recent correction events can be displayed from SQLite.
- AI/context correction has a provider-neutral interface.
- Mock AI provider works for deterministic tests.
- Disabled AI provider works.
- Ollama provider works locally.
- `llama3.2:3b` is the current best local Ollama candidate.
- Prompt tuning and output guardrails are in place.
- Controlled dictionary-first AI-if-needed routing exists.
- AI/context route decisions save route reasons in structured correction records.
- Manual review labels now exist for events.
- Bad automatic corrections can be converted into manual teach-back examples.
- Latest known pytest result: 102 passed.

What is still limited

- Correctr is ready for controlled testing, not broad production use.
- AI corrections can still be plausibly wrong.
- The review workflow is terminal-based.
- No suggestion UI or accept/reject popup exists.
- No undo flow has been implemented beyond normal application-level undo.
- Manual corrections are saved but not yet used live by a personal memory lookup layer.
- Event review labels exist, but no trusted training dataset export exists yet.
- No neural ranker exists yet.
- No packaged/tray app exists yet.
- Clipboard and hotkey behavior still need more cross-app QA.

What is not built yet

- Full spellcheck/candidate generation layer.
- Misplaced-space handler as a dedicated module.
- Personal memory lookup.
- Neural ranker.
- Suggestion UI.
- Polished desktop UI.
- Tray app.
- Production packaging.
- Backup/export management.
- Full README/setup/privacy documentation.
- Decision log/roadmap documents, unless added outside this General Developer sequence.

4. Current Architecture State

The current implementation now maps to the architecture as a partial but working modular foundation.

Hotkey listener

- Implemented in `correctr/hotkey_runner.py`.
- Uses the configured activation hotkey and stop hotkey.
- Triggers the correction workflow without requiring manual copy/paste from the user.
- Current activation hotkey: Ctrl + Alt + C.
- Current stop hotkey: Ctrl + Alt + Q.

Clipboard handler

- Implemented in `correctr/clipboard_handler.py`.
- Sends copy/paste commands.
- Reads selected text from the clipboard.
- Preserves/restores original clipboard content when configured.
- Uses short timing delays for Windows clipboard reliability.

Correction engine

- Implemented in `correctr/correction_engine.py`.
- Owns deterministic local dictionary correction.
- Returns structured `CorrectionResult`.
- Still pure: it does not write to the database and does not call AI.
- Handles basic punctuation and capitalization for known word-level replacements.
- Does not handle full spellcheck, fuzzy matching, or advanced spacing logic.

Correction orchestrator

- Implemented in `correctr/correction_orchestrator.py`.
- Coordinates dictionary correction and AI/context correction.
- Supports multiple pipeline modes:
 - `dictionary_only`
 - `dictionary_then_ai_if_unchanged`
 - `dictionary_then_ai_always`
 - `dictionary_then_ai_if_needed`
- Current preferred controlled test mode: `dictionary_then_ai_if_needed`.
- Default mode remains `dictionary_only`.
- Contains `should_route_to_ai()` and suspicious text signal detection.

AI/context layer

- Implemented in `correctr/llm_engine.py`.
- Provider-neutral interface.
- Supports:
 - `disabled`
 - `mock`
 - `ollama`
 - `openai` reserved but not implemented
- Ollama provider uses local API endpoint:
 - `http://localhost:11434/api/generate`
- Uses `stream = false`.
- Uses conservative correction prompt and output guardrails.
- Uses safe fallback/rejection behavior.

Database/storage layer

- Implemented in correctr/database.py.
- Uses Python built-in sqlite3.
- Stores local correction events in data/corrections.sqlite.
- Supports dictionary, AI/context, and manual correction events.
- Now includes review fields and linked teach-back events.
- Must remain private/local and excluded from GitHub.

Manual capture/review

- Manual capture script: scripts/manual_correction_capture.py.
- Review/teach-back script: scripts/review_recent_events.py.
- Recent event inspection script: scripts/show_recent_events.py.
- Manual Rejection Review_v0.1 introduced review labels and manual teach-back from reviewed events.

Modules still placeholder or future

These files exist but should still be treated as placeholder/future unless another workstream changed them outside this handoff:

- correctr/personal_memory.py
- correctr/neural_ranker.py
- correctr/suggestion_ui.py
- correctr/space_correction.py
- correctr/spellcheck_engine.py

5. Current Folder/File Structure

Latest known project structure:

```
Correctr/
├── 1.Docs/
│   ├── Correctr_Project_Architecture_and_Workflow_Document.docx
│   ├── journal.txt
│   ├── Engineering_Department_Doc_Early_Foundation_Sequence_General_Developer_Handoff_Notes.pdf
│   └── Engineering_Department_Doc_Current_General_Developer_Final_Handoff_Notes.md
├── correctr/
│   ├── __init__.py
│   ├── clipboard_handler.py
│   ├── config.py
│   ├── correction_engine.py
│   ├── correction_orchestrator.py
│   ├── database.py
│   ├── hotkey_runner.py
│   ├── llm_engine.py
│   ├── logging_utils.py
│   ├── neural_ranker.py
│   ├── personal_memory.py
│   ├── space_correction.py
│   ├── spellcheck_engine.py
│   └── suggestion_ui.py
└── data/
```

```

|   ├── backups/
|   ├── dictionaries/
|   └── corrections.sqlite
├── scripts/
|   ├── ai_correction_smoke_test.py
|   ├── init_database.py
|   ├── local_model_evaluation.py
|   ├── manual_correction_capture.py
|   ├── ollama_provider_smoke_test.py
|   ├── orchestrator_smoke_test.py
|   ├── review_recent_events.py
|   ├── show_recent_events.py
|   └── smoke_test_correction.py
├── tests/
|   ├── test_ai_correction_integration.py
|   ├── test_ai_output_guardrails.py
|   ├── test_correction_engine.py
|   ├── test_correction_event_integration.py
|   ├── test_correction_orchestrator.py
|   ├── test_database.py
|   ├── test_llm_engine.py
|   └── test_ollama_provider.py
├── .env.example
├── .gitignore
├── app.py
├── pytest.ini
├── README.md
└── requirements.txt

```

Active files

- `app.py`: current app entry point and hotkey workflow coordinator.
- `correctr/config.py`: runtime config values, pipeline modes, provider choices, Ollama defaults.
- `correctr/hotkey_runner.py`: global hotkey listener.
- `correctr/clipboard_handler.py`: selected text copy/paste workflow.
- `correctr/correction_engine.py`: deterministic dictionary correction and `CorrectionResult`.
- `correctr/correction_orchestrator.py`: routing layer between dictionary and AI/context correction.
- `correctr/llm_engine.py`: provider-neutral AI/context layer, mock/disabled/Ollama providers, prompt/guardrails.
- `correctr/database.py`: SQLite schema, event saving/fetching, review labels, teach-back support.
- `correctr/logging_utils.py`: console logging setup.
- `scripts/`: repeatable project-useful scripts.
- `tests/`: regression tests.
- `pytest.ini`: required for stable pytest import behavior.

Placeholder/future files

- `correctr/personal_memory.py`
- `correctr/neural_ranker.py`
- `correctr/suggestion_ui.py`
- `correctr/space_correction.py`
- `correctr/spellcheck_engine.py`

Do not assume these are meaningfully implemented unless inspected in the current codebase.

6. Workstream-by-Workstream Summary

6.1 Hotkey Clipboard_v0.1

Purpose: Prove the physical workflow: highlight text -> press hotkey -> copy selected text -> Python receives text -> paste replacement back.

Files involved:

- app.py
- correctr/config.py
- correctr/hotkey_runner.py
- correctr/clipboard_handler.py
- correctr/correction_engine.py
- correctr/logging_utils.py

Behavior added:

- Configured hotkey listener.
- Clipboard copy/paste handler.
- Placeholder correction initially returned [Correctr test] original selected text.
- Safe stop hotkey.
- Logging around workflow.

User test result:

- User confirmed highlighted text was replaced in real testing.
- Example placeholder output worked.

Known limitations:

- Clipboard timing can vary by app.
- Triple-click selection can include trailing newline; the system pastes what is selected.
- Not guaranteed across every Windows app.
- No native Windows Text Services Framework integration.

6.2 Correction Engine_v0.2 Basic Local Corrections

Purpose: Replace placeholder correction behavior with simple deterministic local typo correction.

Files involved:

- correctr/correction_engine.py
- tests/test_correction_engine.py
- scripts/smoke_test_correction.py

Behavior added:

- Known typo dictionary.
- Word-level replacements.
- Basic punctuation preservation.
- Basic capitalization preservation.

Examples:

```
testig -> testing
```

```

misakes -> mistakes
hopfully -> hopefully
maknig -> making
sentencs -> sentences
somehting -> something
standard -> standard
nural -> neural
contexr -> context
highlight -> highlight

```

User test result:

- Sample sentence corrected successfully.
- Hotkey workflow worked with the real correction engine.

Known limitations:

- No fuzzy matching.
- No advanced spellcheck.
- No misplaced-space correction.
- Cannot correct many realistic user mistakes by itself.

6.3 Correction Engine_v0.3 Structured CorrectionResult

Purpose: Prepare correction output for future database logging, personal memory, AI/context scoring, and neural ranker training.

Files involved:

- correctr/correction_engine.py
- tests/test_correction_engine.py
- scripts/smoke_test_correction.py

Behavior added:

- CorrectionResult structure with:
 - original_text
 - corrected_text
 - changed
 - corrections
 - engine_version
- correct_text(text) still returns a plain string.
- correct_text_detailed(text) returns structured result.

Known limitations:

- Correction records are still local-dictionary based.
- Correction engine intentionally remains pure and does not save to database.

6.4 Manual Correction Capture_v0.1

Purpose: Capture manual original/corrected pairs as useful local training data.

Files involved:

- correctr/database.py

- scripts/manual_correction_capture.py
- tests/test_database.py

Behavior added:

- Terminal script asks for original bad text, corrected text, and optional note.
- Saves source as manual.
- Engine version: manual_capture_v0.1.

User test result:

- Manual capture worked.
- Example manual correction was saved.

Known limitations:

- Terminal-only.
 - Manual corrections are stored but not yet used by live correction logic.
-

6.5 Correction Event Integration_v0.1

Purpose: Connect structured CorrectionResult from the correction engine to SQLite event logging.

Files involved:

- app.py
- correctr/database.py
- scripts/show_recent_events.py
- tests/test_correction_event_integration.py
- pytest.ini

Behavior added:

- Dictionary correction events save from app/hotkey workflow.
- Changed events save with source dictionary.
- No-change events are skipped.
- scripts/show_recent_events.py displays saved events.

User test result:

- User confirmed dictionary and manual events were visible in recent events.

Known limitations:

- Early database contained mixed test/dev data.
 - No review labels existed yet at this stage.
-

6.6 AI Context Correction_v0.1 Mock Provider

Purpose: Add a provider-neutral AI/context interface without requiring API keys, Ollama, or real model setup.

Files involved:

- correctr/llm_engine.py
- correctr/config.py
- correctr/database.py
- scripts/ai_correction_smoke_test.py

- tests/test_llm_engine.py
- tests/test_ai_correction_integration.py

Behavior added:

- AI provider modes:
- mock
- disabled
- future reserved provider values
- Mock provider corrects known hard examples.
- AI/context results can be saved as source ai_context.
- Disabled provider returns no-change result.

User test result:

- Smoke test passed.
- pytest passed after Protocol import bug was fixed.

Known limitations:

- Mock provider is deterministic and not intelligent.
- App/hotkey workflow was not changed at this stage.

6.7 Correction Orchestrator_v0.1

Purpose: Create a routing layer above dictionary correction and AI/context correction.

Files involved:

- correctr/correction_orchestrator.py
- app.py
- correctr/config.py
- scripts/orchestrator_smoke_test.py
- tests/test_correction_orchestrator.py

Behavior added:

Pipeline modes:

- dictionary_only
- dictionary_then_ai_if_unchanged
- dictionary_then_ai_always

App now calls the orchestrator instead of directly deciding dictionary vs AI.

User test result:

- App/hotkey workflow worked through orchestrator.
- Recent events showed source = dictionary and source = ai_context as expected.

Known limitations:

- dictionary_then_ai_if_unchanged was too conservative.
- dictionary_then_ai_always was unsafe with real local models.

6.8 AI Context Correction_v0.2 Ollama Provider Prototype

Purpose: Add a real local-model provider path through Ollama without making it default.

Files involved:

- correctr/llm_engine.py
- correctr/config.py
- scripts/ollama_provider_smoke_test.py
- tests/test_llm_engine.py
- tests/test_ollama_provider.py

Behavior added:

- Ollama provider.
- Default base URL: http://localhost:11434.
- Default model initially: llama3.2:1b.
- Timeout support.
- Fallback when Ollama unavailable.
- Non-streaming API request with stream = false.

User test result:

- When Ollama was not running, provider safely fell back.
- After Ollama install and model pull, provider ran locally.
- pytest passed after prompt string syntax hotfix.

Known limitations:

- llama3.2:1b quality was too weak for normal use.
- No real OpenAI provider added.

6.9 AI Context Correction_v0.3 Prompt Tuning + Output Guardrails

Purpose: Improve local model behavior and prevent bad formatted model outputs from being pasted.

Files involved:

- correctr/llm_engine.py
- tests/test_llm_engine.py
- tests/test_ollama_provider.py
- tests/test_ai_output_guardrails.py
- scripts/ollama_provider_smoke_test.py
- scripts/local_model_evaluation.py

Behavior added:

- Stronger conservative correction prompt.
- Compact examples.
- Output cleanup:
- strips Here is the corrected text:
- strips Corrected text:
- strips Correction:
- strips surrounding quotes
- handles basic sentence-start capitalization
- Output validation:

- rejects empty output
- rejects code fences
- rejects assistant-style explanations
- rejects excessive length
- rejects multiple paragraphs when original was one sentence
- Rejected outputs become safe no-change `CorrectionResult`.

User test result:

- llama3.2:1b improved but remained weak.
- pytest passed after a guardrail test-order hotfix.

Known limitations:

- Guardrails catch formatting failures, not all semantic failures.
- Plausible but wrong corrections remain possible.

6.10 Local Model Evaluation_v0.2 Stronger Ollama Model Comparison

Purpose: Compare local models using the same prompt contract, guardrails, and evaluation cases.

Files involved:

- scripts/local_model_evaluation.py

Models tested:

- llama3.2:1b
- llama3.2:3b
- qwen2.5:3b
- gemma3:4b

Findings:

- llama3.2:1b: usable on simple examples but too weak for hard examples.
- llama3.2:3b: best current candidate but made at least one false correction on clean text.
- qwen2.5:3b: good on some cases but overcorrected/dropped wording.
- gemma3:4b: good quality on some cases but too slow/risky due to timeout.

Current model decision:

- llama3.2:3b is the current best local candidate.
- It is not safe for blind AI-always use.

Known limitations:

- Evaluation set is small.
- User manually interprets quality.
- More real-use testing is needed.

6.11 Controlled App Routing_v0.2

Purpose: Add a safer routing policy: dictionary first, AI/context only if needed, skip AI for obviously clean text.

Files involved:

- correctr/config.py

- correctr/correction_orchestrator.py
- scripts/orchestrator_smoke_test.py
- tests/test_correction_orchestrator.py
- tests/test_ollama_provider.py

Behavior added:

New mode:

```
dictionary_then_ai_if_needed
```

Current routing logic checks for known hard typo tokens and split-word patterns such as:

```
shiuld
thsi
snother
pracitve
sentenc
habe
testin
gexample
abl eto
mis takes
i ma
th e
th ethird
testin gexample
```

Reason codes include:

```
clean_text_gate_skipped_ai
dictionary_no_change_but_suspicious_pattern
dictionary_changed_but_residual_suspicious_pattern
dictionary_changed_no_residual_suspicious_pattern
```

User test result:

- Real events showed source ai_context, engine orchestrator_v0.2:ai_context, model llama3.2:3b, and route reasons.
- Good correction example: Thsi here be th ethird pracitve testing sentenc -> This here be the third practice testing sentence.
- Bad correction example remained: Laso... -> Lasso... when intended was Also....

Known limitations:

- Routing is rule-based and narrow.
- Clean-text detection is not intelligent.
- AI can still make plausible wrong corrections.
- No accept/reject UI exists.

6.12 Manual Rejection Review_v0.1 Recent Event Review + Teach-Back

Purpose: Add trust/data-quality review labels and a way to turn bad automatic corrections into manual teach-back examples.

Files involved:

- correctr/database.py

- scripts/review_recent_events.py
- scripts/show_recent_events.py
- tests/test_database.py

Behavior added:

Database review fields:

```
review_status
reviewed_at
linked_event_id
review_notes
```

Review statuses:

```
unreviewed
accepted
rejected
manually_corrected
test_event
uncertain
```

New functions:

```
fetch_correction_event_by_id()
mark_correction_event_reviewed()
save_manual_correction_from_event()
```

Script behavior:

- Show recent events.
- Select event by ID.
- Mark accepted.
- Mark rejected.
- Enter intended correction/manual teach-back.
- Mark test/development event.
- Mark uncertain.
- Cancel.

User test result:

- Safe migration worked.
- Existing events now show Review status: unreviewed.
- Latest pytest result: 102 passed.

Known limitations:

- Terminal-only.
- Reviews one event at a time.
- Cannot yet link an already-existing manual event to a bad automatic event without creating a new teach-back event.
- Future memory/ranker/export logic does not yet consume review labels.

7. Current Configuration State

Current config is in correctr/config.py.

Hotkey configuration

```
activation_hotkey = "<ctrl>+<alt>+c"
stop_hotkey = "<ctrl>+<alt>+q"
```

Timing/safety configuration

Configured in AppConfig:

```
hotkey_release_delay_seconds
clipboard_poll_timeout_seconds
clipboard_poll_interval_seconds
before_copy_delay_seconds
before_paste_delay_seconds
after_paste_delay_seconds
restore_original_clipboard_after_paste
```

AI provider options

```
AI_PROVIDER = "mock"
ALLOWED_AI_PROVIDERS = {"disabled", "mock", "ollama", "openai"}
```

Current default remains mock. openai is reserved and not implemented.

Ollama configuration

```
OLLAMA_BASE_URL = "http://localhost:11434"
OLLAMA_MODEL = "llama3.2:3b"
OLLAMA_TIMEOUT_SECONDS = 10.0
```

llama3.2:3b is the current preferred local model candidate from the evaluation sequence.

Pipeline modes

```
CORRECTION_PIPELINE_MODE = "dictionary_only"
ALLOWED_CORRECTION_PIPELINE_MODES = {
    "dictionary_only",
    "dictionary_then_ai_if_unchanged",
    "dictionary_then_ai_always",
    "dictionary_then_ai_if_needed",
}
```

Recommended modes

Safest stable default:

```
CORRECTION_PIPELINE_MODE = "dictionary_only"
```

Controlled AI testing mode:

```
CORRECTION_PIPELINE_MODE = "dictionary_then_ai_if_needed"
AI_PROVIDER = "ollama"
OLLAMA_MODEL = "llama3.2:3b"
```

Do **not** leave dictionary_then_ai_always on blindly. The local model can still make plausible wrong corrections.

8. Current Database State and Schema

Database path:

```
data/corrections.sqlite
```

This file must remain local/private and excluded from GitHub.

Table: **correction_events**

Current fields:

```
id
created_at
source
original_text
corrected_text
changed
corrections_json
engine_version
notes
review_status
reviewed_at
linked_event_id
review_notes
```

Source values

Current allowed sources:

```
dictionary
manual
ai_context
future_llm
future_memory
future_ranker
```

Review status values

```
unreviewed
accepted
rejected
manually_corrected
test_event
uncertain
```

linked_event_id behavior

When a bad automatic event is converted into a manual teach-back event:

- Original automatic event can be marked `manually_corrected`.
- New manual event has `source = manual`.
- New manual event stores the original text from the automatic event.
- New manual event stores the user-provided intended correction.
- New manual event can store `linked_event_id = original_bad_event_id`.

What gets saved

Dictionary corrections:

- Saved when changed.
- `source = dictionary`.
- Structured correction records included.
- No-change dictionary results are skipped.

AI/context corrections:

- Saved when accepted final orchestrator result changed.
- `source = ai_context`.
- Structured correction records can include provider, model, pipeline stage, and route reasons.
- Fallback/rejected/no-change AI results are not useful positive examples and should not be treated as trusted.

Manual corrections:

- Saved through `scripts/manual_correction_capture.py` or `scripts/review_recent_events.py`.
- `source = manual`.
- Intended as the cleanest teach-back data, but still should be reviewable.

Current data-quality status

The current database contains mixed-quality development/test data:

- Good dictionary corrections.
- Good AI corrections.
- Bad AI corrections.
- Duplicate tests.
- Mock-provider examples.
- llama3.2:1b experiments.
- Manual examples.
- Unreviewed rows.

Do not treat unreviewed events as positive training data.

9. Current Scripts and Commands

Core app command

```
python app.py
```

Runs Correctr hotkey app.

Test command

```
pytest
```

Runs the full test suite.

Script inventory

```
scripts/init_database.py
```

Initializes SQLite database and schema.

```
scripts/manual_correction_capture.py
```

Manually enters original/corrected text and saves a source manual event.

```
scripts/show_recent_events.py
```

Displays recent database correction events, including review status after Manual Rejection Review_v0.1.

```
scripts/review_recent_events.py
```

Terminal review workflow for labeling events and saving manual teach-back corrections.

```
scripts/smoke_test_correction.py
```

Runs local dictionary correction smoke test.

```
scripts/ai_correction_smoke_test.py
```

Runs mock AI/context correction smoke test.

```
scripts/ollama_provider_smoke_test.py
```

Tests Ollama provider path and reports fallback/rejection status.

```
scripts/local_model_evaluation.py
```

Runs evaluation cases across configured/local Ollama model(s). Supports model comparison.

```
scripts/orchestrator_smoke_test.py
```

Tests orchestrator routing modes and decision reasons.

10. Current Test Status

Latest known pytest result:

```
102 passed in 1.13s
```

Important test files:

```
tests/test_correction_engine.py
tests/test_database.py
tests/test_correction_event_integration.py
tests/test_ai_correction_integration.py
tests/test_llm_engine.py
tests/test_ollama_provider.py
tests/test_ai_output_guardrails.py
tests/test_correction_orchestrator.py
```

pytest.ini must stay in the repo:

```
[pytest]
testpaths = tests
pythonpath = .
```

Without pytest.ini, tests may fail with:

```
ModuleNotFoundError: No module named 'correctr'
```

Tests should use temporary database paths and must not write to data/corrections.sqlite.

11. Model Evaluation Summary

llama3.2:1b

Result: Too weak for normal use.

Strengths:

- Good on simple typo and spacing examples after prompt tuning.
- Low resource demand.

Failures:

- Failed harder typo cases.
- Did not fix examples such as Snother testin gexample.
- Left or created poor outputs in earlier tests.

llama3.2:3b

Result: Current best local candidate.

Strengths:

- Corrected the key hard examples better than llama3.2:1b.
- Corrected Snother testin gexample. to Another testing example. in model comparison.
- Worked well on phrase-level spacing cases.
- Reasonable enough for controlled routing tests.

Failure/risk:

- Falsely changed an already-correct sentence in evaluation:
- This is already correct.
- became This is alreedy correct.
- Still produced plausible wrong correction during app testing:
- Laso...
- became Lasso...
- intended was Also...

Conclusion:

- Use only with controlled routing and review workflow.
- Do not use blind AI-always mode.

qwen2.5:3b**Result:** Not preferred.**Strengths:**

- Good on several basic examples.

Failures:

- Dropped wording or shifted meaning.
- Example: Snother testin gexample. became Another example., losing testing.
- Example: practice became practical in the style-preserving test.

gemma3:4b**Result:** Quality promising but too slow/risky for current default.**Strengths:**

- Good on several examples.

Failures:

- Timed out on at least one case.
- Larger/slower than necessary for current phase.

Current decision

Use llama3.2:3b as the current local Ollama test model, but only through controlled routing and review.

12. Important Decisions Made

- Keep correctr/correction_engine.py pure.
- Use CorrectionResult as the shared result structure.
- Use local-first SQLite storage.
- Keep data/corrections.sqlite out of GitHub.

- Skip no-change events under the current event logging policy.
- Use source labels:
- dictionary
- manual
- ai_context
- Use provider-neutral AI/context design.
- Use Ollama as the first local model provider.
- Keep OpenAI reserved but not implemented.
- Use controlled AI routing, not AI-always default.
- Use llama3.2:3b as current best local model candidate.
- Add output guardrails before using local model output.
- Add event review labels before personal memory or neural ranker training.
- Avoid temporary instruction files in repo packages.
- Use zip assignment packages for meaningful code work.
- Put unload/test instructions in chat, not repo clutter.
- Handoff documents should be created at meaningful checkpoints.
- Do not start neural ranker until data quality/export logic is strong enough.

13. Known Risks and Limitations

AI plausible wrong corrections

Local model output can look plausible but be wrong. Example:

Original:

Laso I ma making many examlpe sentnces.

AI output:

Lasso I am making many example sentences.

Intended:

Also I am making many example sentences.

Mixed-quality database

The database now includes test, duplicate, good, bad, manual, mock, and local model events. Review labels are now present, but most rows remain unreviewed.

Local model latency/quality variation

- llama3.2:3b is useful but not perfect.
- gemma3:4b timed out in testing.
- Larger models are not automatically better if they are too slow.

No suggestion UI yet

The user currently cannot accept/reject correction through a polished popup. Review happens after the fact through terminal scripts.

No undo workflow yet

The user can rely on target-app undo behavior, but Correctr has no built-in undo/restore feature yet.

No personal memory lookup yet

Manual and reviewed corrections are saved but not yet used live.

No neural ranker yet

The neural ranker is not started. It should not be started until trusted examples can be exported.

No production packaging

The app runs from terminal. No tray app, installer, packaged executable, or startup behavior exists yet.

Clipboard/hotkey fragility

Clipboard automation may vary by app, focus state, and selection behavior. Needs QA across Notepad, browser text boxes, ChatGPT input, VS Code, and other common targets.

Schema migration caution

Database migration is currently simple and safe, but future schema changes should be cautious because the local database contains personal text.

14. What Should Happen Next

Recommended next sequence:

1. Testing / QA Assistant foundation review

Have QA review:

- hotkey/clipboard workflow
- app behavior across apps
- current tests
- event logging
- review workflow
- privacy/.gitignore coverage
- schema migration assumptions
- current routing behavior

2. New developer chat onboarding

Use this handoff to onboard the next developer or specialist. The next developer should not need full chat history.

3. Personal Memory Lookup_v0.1

Build a small memory lookup layer that can retrieve trusted manual or accepted corrections.

Do not use unreviewed AI events as trusted examples.

4. Data Quality / Training Dataset Export_v0.1

Export only trusted examples:

- source = manual
- review_status = accepted
- review_status = manually_corrected
- maybe selected accepted dictionary/AI events

Exclude:

- rejected
- test_event
- uncertain
- unreviewed AI events
- known bad older model experiments

5. Neural Ranker_v0.1 baseline

Only start after event review and export logic are good enough.

The neural ranker should be a candidate ranking helper, not a full correction model.

15. What Future Developers Should Read First

Future developers and QA assistants should read:

1. Correctr_Project_Architecture_and_Workflow_Document.docx
2. This handoff:
 - Engineering_Department_Doc_Current_General_Developer_Final_Handoff_Notes.md
3. Early foundation handoff:
 - Engineering_Department_Doc_Early_Foundation_Sequence_General_Developer_Handoff_Notes.pdf
4. app.py
5. correctr/config.py
6. correctr/correction_engine.py
7. correctr/correction_orchestrator.py
8. correctr/llm_engine.py
9. correctr/database.py
10. scripts/
11. tests/
12. Latest pytest result.
13. Latest scripts/show_recent_events.py output if debugging data quality.

16. What Future Developers Should Not Assume

Do not assume:

- AI outputs are always correct.
- Unreviewed events are positive training examples.
- Manual corrections are already used in live correction.
- The neural ranker exists.
- The suggestion UI exists.
- The database schema is final.
- Larger models are automatically better.
- dictionary_then_ai_always is safe.
- The app is production-ready.
- Every saved correction event should be used for training.
- Old mock/llama3.2:1b events are real quality examples.
- The current suspicious-pattern list is complete.
- The current prompt is final.

- The current database is clean.
 - The review workflow is polished.
 - The app works equally well in every Windows application.
-

17. Final Handoff Summary

The current General Developer phase established Correctr's working local foundation.

What has been built:

- Hotkey/clipboard app loop.
- Local dictionary correction.
- Structured `CorrectionResult`.
- SQLite correction event storage.
- Manual correction capture.
- Dictionary and AI/context event logging.
- Provider-neutral AI/context layer.
- Mock, disabled, and Ollama providers.
- Local Ollama testing.
- Prompt tuning and output guardrails.
- Model comparison script.
- Controlled dictionary-first AI-if-needed routing.
- Review labels and manual rejection/teach-back workflow.
- Regression test suite with latest known result of 102 passed.

Why it matters:

- Correctr can now be tested as a real local workflow.
- The system can collect correction data.
- The project has a path toward personal memory and eventual model/ranker work.
- Bad AI corrections can now be identified and converted into useful manual examples instead of silently polluting training data.

What is safe to continue from:

- The modular code structure.
- `CorrectionResult`.
- SQLite event schema with review labels.
- Controlled routing approach.
- Ollama provider architecture.
- Local model evaluation script.
- Existing tests and smoke scripts.

What should be reviewed before the next phase:

- QA should inspect app behavior and database/review workflow.
- Lead Developer should confirm the next workstream.
- Do not start neural ranker work until trusted data export is built.
- Do not assume unreviewed AI events are usable training data.

This handoff is ready for Lead Developer review.